

TRƯỜNG ĐẠI HỌC BÁCH KHOA
KHOA: CÔNG NGHỆ THÔNG TIN
BỘ MÔN: CÔNG NGHỆ PHẦN MỀM
BÀI LÀM THI CUỐI KỲ

Tên học phần: Toán ứng dụng CNTT

Mã học phần:

Phương pháp đánh giá: Tự luận có giám sát

Đề số: **T000.0002**

Mã số sinh viên: 102210153

Họ và tên: Nguyễn Đoàn Bảo Châu

Lớp học phần: 21Nh99

Lớp sinh hoạt: 21TCLC-DT2

Số tín chỉ: **03**

Thời gian làm bài: **90** phút

Câu 1 (2 điểm) Cho hệ đồng dư sau:

$$\begin{cases} x \equiv 2 \pmod{5} \\ x \equiv 4 \pmod{7} \\ x \equiv 6 \pmod{9} \\ x \equiv 8 \pmod{11} \\ x \equiv 9 \pmod{13} \end{cases}$$

a) (1 điểm) Trình bày phương pháp giải hệ phương trình đồng dư đã cho.

Trả lời: trình bày sơ đồ khối hoặc ngôn ngữ tự nhiên:

Input: mảng 1 chiều chứa các giá trị $a_0 \dots a_n$ (1,5,4,2) và 1 mảng chứa giá trị $m_0 \dots m_n$ (3,7,11,13)

Output: nghiệm X của hệ phương trình đồng dư

Thuật toán:

- Bước 1: Nhập vào số phương trình (n) và các giá trị a_i và m_i tương ứng

- Bước 2: Thực hiện tính M và M_i tương ứng (trong hàm Cal_M):

Gán $MT = 1$;

Lặp $i = 1 \rightarrow n$

Gán $MT = MT * m_i$

Lặp $i=1 \rightarrow n$

Gán $M_i = MT/m_i$

- Bước 3: tính các giá trị y_i tương ứng (trong hàm cal_y):

Lặp $i = 1 \rightarrow n$

Gán $y_i = 1$

Lặp khi ($M_i * y_i$) % $m_i \neq 1$

$Y_i = y_i + 1$

- Bước 4: Tính $AT = ($ với I chạy từ 1 đến n)

- Bước 5: In ra kết quả của phương trình

$X = AT * MT \pmod{MT}$

b) (1 điểm) Viết chương trình (có dùng hàm) giải hệ phương trình đồng dư trên.

Trả lời: Dán code vào bên dưới:

```
#include <bits/stdc++.h>
#include<math.h>
#include<vector>
using namespace std;
#define max 100

int findMinX(int m[], int a[], int n)
{
    int x = 1;
    while (true)
    {
        int j;
        for (j=0; j<n; j++ )
            if (x%m[j] != a[j])
                break;
        if (j == n)
            return x;
        x++;
    }

    return x;
}

void nhap(int a[], int m[],int n){
    for(int i = 0; i < n; i++){
        cout << "Nhap a[" << i << "]= " ; cin >> a[i];
        cout << "Nhap m[" << i << "]= " ; cin >> m[i];
    }
}

int main(){
```

```

int a[max], m[max] ;//a mod m

int n; int tich = 1;

cout << "Nhap n = "; cin >> n;

nhap(a, m, n);

for(int i = 0; i < n; i++){

    cout << endl << a[i] << " mod " << m[i];

    tich *= m[i];

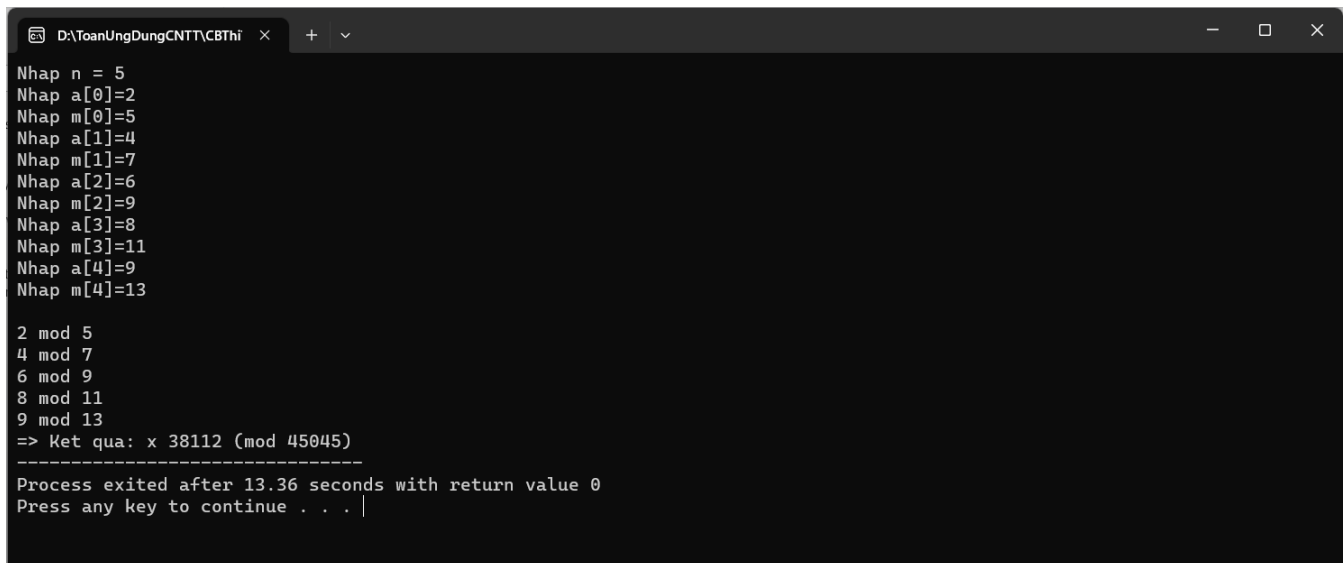
}

cout << endl << "=> Ket qua: x " << findMinX(m,a,n) << " (mod " << tich << ")";

}

```

Trả lời: Dán kết quả thực thi vào bên dưới:



```

D:\ToanUngDungCNTT\CBThi x + v
Nhap n = 5
Nhap a[0]=2
Nhap m[0]=5
Nhap a[1]=4
Nhap m[1]=7
Nhap a[2]=6
Nhap m[2]=9
Nhap a[3]=8
Nhap m[3]=11
Nhap a[4]=9
Nhap m[4]=13

2 mod 5
4 mod 7
6 mod 9
8 mod 11
9 mod 13
=> Ket qua: x 38112 (mod 45045)
-----
Process exited after 13.36 seconds with return value 0
Press any key to continue . . .

```

Câu 2 (2 điểm): Cho ma trận A.

a) (1 điểm) Trình bày phương pháp phân rã ma trận A bằng SVD.

Trả lời: trình bày sơ đồ khối hoặc ngôn ngữ tự nhiên:

- Input: m: số hàng, n: số cột, và giá trị từng phần tử của ma trận a.
- Output: ma trận U, ma trận sigma, ma trận V_T.

+ Bước 1:

Tính ma trận chuyển vị (AT) và tính AT*a và tính giá trị riêng (GTR)

Gán AT = np.transpose (A)

Gán ATA = ATA

Gán GTR,VTR = la.eig(np.array (ATA))

+ Bước 2:

Tính giá trị S (ma trận đường chéo của giá trị riêng) và S1

Gán S = np.diag (GTR)

```

Lặp i = 0 đến n
Nếu S[i][i] != 0
S[i][i] = math.sqrt ( S[i][i] )
S1[i][i] = 1/S[i][i]
+ Bước 3: Tính right singular vector. Lặp I = 0 đến n
Gán sum = 0
Lặp j = 0 đến n
Sum = sum + VTR[i][j] * VTR[i][j]
X = math.sqrt ( sum )
Lặp j = 0 đến n
VTR[i][j] = VTR[i][j]/x
+ Bước 4: Tính left singular vector
U = A.transpose(VTR)
U = U.S1

```

b) (1 điểm) Viết chương trình (có dùng hàm) phân rã ma trận A bằng SVD.

Trả lời: Dán code vào bên dưới:

```

#include <bits/stdc++.h>

#include <Eigen/Dense>
#include <Eigen/Eigenvalues>

using namespace Eigen;
using namespace std;

float data[100000];

int rows, cols;

struct mat
{
    int sd, sc;

    float coef[100][100];
};

struct mat createMatrix(int m, int n)
{
    struct mat A;

    int i, j;

```

```

        A.sd = m;

        A.sc = n;

        for (i = 0; i < m; i++)
            for (j = 0; j < n; j++)
                A.coef[i][j] = 0;

        return A;
    }

int Enter()
{
    cout << "Enter number of row and column: ";
    cin >> rows >> cols;
    cout << "Enter Matrix A: " << endl;
    for (int i = 0; i < rows * cols; i++)
    {
        cin >> data[i];
    }
}

void SVD()
{
    MatrixXf matrix(rows, cols);
    for (int i = 0; i < rows; i++)
    {
        for (int j = 0; j < cols; j++)
        {
            matrix(i, j) = data[i * cols + j];
        }
    }
}

```

```

struct mat sig = createMatrix(rows, cols);
MatrixXf sigma(rows, cols);
MatrixXf matrixA = matrix.transpose() * matrix; //A_T*A

SelfAdjointEigenSolver<MatrixXf> esA(matrixA);
int k = 0;
VectorXf temp = esA.eigenvalues(); //temp: gia tri rieng

for (int i = 0; i < temp.size(); i++)
{
    if (fabs(temp[i]) < 10e-5)
        temp[i] = 0;
    temp[i] = sqrt(temp[i]);
    sig.coef[temp.size() - i - 1][temp.size() - i - 1] = temp[i];
}

MatrixXf V = esA.eigenvectors().transpose();
struct mat Vt = createMatrix(cols, cols);
for (int i = 0; i < cols; i++)
{
    for (int j = 0; j < cols; j++)
    {
        Vt.coef[cols - i - 1][j] = V(i, j);
        cout << "V = " << V(i,j);
    }
}

MatrixXf matrixB = matrix * matrix.transpose();
SelfAdjointEigenSolver<MatrixXf> esB(matrixB);

```

```
MatrixXf U = esB.eigenvectors();
```

```
for (int i = 0; i < U.cols() / 2; i++)  
{  
    temp = U.col(U.cols() - i - 1);  
    U.col(U.cols() - i - 1) = U.col(i);  
    U.col(i) = temp.col(0);  
}
```

```
for (int i = 0; i < cols; i++)  
{  
    for (int j = 0; j < cols; j++)  
    {  
        V(i, j) = Vt.coef[i][j];  
        if (fabs(V(i, j)) < 10e-5)  
            V(i, j) = 0;  
    }  
}
```

```
for (int i = 0; i < rows; i++)  
{  
    for (int j = 0; j < cols; j++)  
    {  
        sigma(i, j) = sig.coef[i][j];  
        if (fabs(sigma(i, j)) < 10e-5)  
            sigma(i, j) = 0;  
    }  
}
```

```
for (int i = 0; i < min(sigma.rows(), sigma.cols()); i++)
```

```

{
    if (sigma(i, i) != 0)
    {
        temp = V.row(i);
        temp = temp.transpose();
        temp = matrix * temp / sigma(i, i);
        U.col(i) = temp.col(0);
    }
}

for (int i = 0; i < rows; i++)
{
    for (int j = 0; j < rows; j++)
    {
        if (fabs(U(i, j)) < 10e-5)
            U(i, j) = 0;
    }
}

cout << "SVD Decomposition  $A = U * \text{Sigma} * V^T$  :" << endl;
cout << setprecision(5) << fixed;
cout << "\nU\n\n" << U << endl;
cout << "\nSigma\n\n" << sigma << endl;
cout << "\nV^T\n\n" << V << endl;
cout << "Thu lai" << endl;
MatrixXf mtKQ ;
if (V.cols() < U.cols()){
    MatrixXf mtVnew(rows, cols);
    for (int i = 0; i < V.cols(); i++)
    {
        for (int j = 0; j < U.cols(); j++)
        {

```

```

        if (j == U.cols()-1)
            mtVnew(i, j) = 0;
        else mtVnew(i, j) = V(i,j);
    }
}
mtKQ = U*sigma*mtVnew;
}
else    mtKQ = U*sigma*V;

for (int i = 0; i < cols; i++)
{
    for (int j = 0; j < cols; j++)
    {
        if (fabs(mtKQ(i, j)) < 10e-5)
            mtKQ(i, j) = 0;
        cout << setw(7) << setprecision(4) << fixed << mtKQ(i,j) << "\t";
    }
    cout << endl;
}
}

int main()
{
    Enter();
    SVD();
    return 0;
}

```

Trả lời: Dán kết quả thực thi vào bên dưới, biết rằng

$$A = \begin{bmatrix} 2 & 4 & 6 \\ 4 & 6 & 2 \\ 3 & 3 & 3 \end{bmatrix}, \text{ sai số } \varepsilon = 10^{-5}.$$

```

U
0.62902  0.70711 -0.32301
0.62902 -0.70711 -0.32301
0.45680  0.00000  0.88957

Sigma
11.23138  0.00000  0.00000
0.00000  3.46410  0.00000
0.00000  0.00000  0.92529

V^T
0.45805  0.68207  0.57006
-0.40825 -0.40825  0.81650
0.78964 -0.60672  0.09146

Thu lại
2.0000  4.0000  6.0000
4.0000  6.0000  2.0000
3.0000  3.0000  3.0000
-----

```

Câu 3 (3 điểm): Cho 15 điểm trong không gian Oxy như sau: (1, 3); (3,1); (5,3); (6,5); (7,6); (3,2); (2,5); (2,9); (4,7); (6,3); (7,9); (6,1); (3,10); (5,9); (7,13)

a) (1.0 điểm) Mô tả thuật toán xác định bao lồi

Trả lời: Dán sơ đồ khối hoặc ngôn ngữ tự nhiên vào bên dưới:

Input: Các điểm trong không gian Oxy: (1, 3); (3,1); (5,3); (6,5); (7,6); (3,2); (2,5); (2,9); (4,7); (6,3); (7,9); (6,1); (3,10); (5,9); (7,13)

Output: các tập điểm của bao lồi

+ Bước đầu:

Chọn P là điểm trái nhất trong tập các điểm (để đảm bảo P nằm trong tập bao lồi).

Chọn hướng ta đang nhìn $\vec{v} \rightarrow$ là từ hướng dưới lên trên.

+ Tiếp theo, các bước sẽ lặp lại đến khi tìm được bao lồi:

Ta quay $\vec{v} \rightarrow$ theo chiều kim đồng hồ cho đến khi ta nhìn thấy một điểm, gọi điểm đó là Q.

Ta cầm theo dải băng và đi đến điểm Q.

Khi ta đến Q, ta thay: $\vec{v} \rightarrow$ thành $PQ \rightarrow$

P thành Q

+ Thuật toán kết thúc khi ta trở về điểm ban đầu. Lúc này ta đã đi đến tất cả các đỉnh của bao lồi theo chiều kim đồng hồ.

b) (1.0 điểm) Viết chương trình (có dùng hàm) xác định bao lồi và tính diện tích bao lồi tìm được.

Trả lời: Dán code bên dưới:

```
#include <bits/stdc++.h>
```

```

using namespace std;

struct Point
{
double x, y;
};

double polygonArea(vector<Point> P, int n)
{
double area = 0.0;
int j = n - 1;
for (int i = 0; i < n; i++)
{
area += (P[j].x + P[i].x) * (P[j].y - P[i].y);
j = i;
}
return abs(area / 2.0);
}

int orientation(Point p, Point q, Point r)
{
double val = (q.y - p.y) * (r.x - q.x) - (q.x - p.x) * (r.y - q.y);
if (val == 0) return 0;
return (val > 0)? 1: 2;
}

void convexHull(Point points[], int n)
{
if (n < 3) return;
vector<Point> hull;
int l = 0;
for (int i = 1; i < n; i++)
if (points[i].x < points[l].x)
l = i;

```

```

int p = l, q;
do
{
hull.push_back(points[p]);
q = (p + 1) % n;
for (int i = 0; i < n; i++)
{
if (orientation(points[p], points[i], points[q]) == 2)
q = i;
}
p = q;
} while (p != l);

cout << "Cac tap diem cua bao loi: ";
for (int i = 0; i < hull.size(); i++)
cout << "(" << hull[i].x << ", " << hull[i].y << ")", ";
cout << endl;

cout << "Dien tich cua bao loi: " << polygonArea(hull, hull.size()) << endl;
}

int main()
{
Point points[] = {{1, 3}, {3,1}, {5,3}, {6,5},{7,6},
                  {3,2}, {2,5}, {2,9}, {4,7}, {6,3},
                  {7,9}, {6,1}, {3,10}, {5,9}, {7,13}};

int n = sizeof(points)/sizeof(points[0]);
convexHull(points, n);
return 0;
}

```

Trả lời: Dán kết quả thực thi vào bên dưới:

```
D:\ToanUngDungCNTT\CBThi x + v
Cac tap diem cua bao loi: (1, 3), (3, 1), (6, 1), (7, 6), (7, 9), (7, 13), (3, 10), (2, 9),
Dien tích của bao loi: 51

-----
Process exited after 0.03542 seconds with return value 0
Press any key to continue . . . |
```

- c) (1.0 điểm) Viết chương trình (có dùng hàm) xác khoảng cách ngắn nhất giữa các đỉnh của bao lồi tìm được đảm bảo độ phức tạp ***nlogn***.

Trả lời: Dán code bên dưới:

```
#include <bits/stdc++.h>

using namespace std;

class Point
{
public:
int x, y;
};

int compareX(const void* a, const void* b)
{
Point *p1 = (Point *)a, *p2 = (Point *)b;
return (p1->x - p2->x);
}

int compareY(const void* a, const void* b)
{
Point *p1 = (Point *)a, *p2 = (Point *)b;
return (p1->y - p2->y);
}

float dist(Point p1, Point p2)
{
return sqrt( (p1.x - p2.x)*(p1.x - p2.x) +
(p1.y - p2.y)*(p1.y - p2.y)
);
}
```

```

}

float bruteForce(Point P[], int n)
{
    float min = FLT_MAX;
    for (int i = 0; i < n; ++i)
        for (int j = i+1; j < n; ++j)
            if (dist(P[i], P[j]) < min)
                min = dist(P[i], P[j]);
    return min;
}

float min(float x, float y)
{
    return (x < y)? x : y;
}

float stripClosest(Point strip[], int size, float d)
{
    float min = d;
    qsort(strip, size, sizeof(Point), compareY);
    for (int i = 0; i < size; ++i)
        for (int j = i+1; j < size && (strip[j].y - strip[i].y) < min; ++j)
            if (dist(strip[i], strip[j]) < min)
                min = dist(strip[i], strip[j]);
    return min;
}

float closestUtil(Point P[], int n)
{
    if (n <= 3)
        return bruteForce(P, n);
    int mid = n/2;
    Point midPoint = P[mid];

```

```

float dl = closestUtil(P, mid);
float dr = closestUtil(P + mid, n - mid);
float d = min(dl, dr);
Point strip[n];
int j = 0;
for (int i = 0; i < n; i++)
if (abs(P[i].x - midPoint.x) < d)
strip[j] = P[i], j++;
return min(d, stripClosest(strip, j, d) );
}
float closest(Point P[], int n)
{
qsort(P, n, sizeof(Point), compareX);
return closestUtil(P, n);
}
int main()
{
Point P[] = {{1, 3}, {3,1}, {7,6},{2,9}, {7,9}, {6,1}, {3,10}, {7,13}};
int n = sizeof(P) / sizeof(P[0]);
cout << "The smallest distance is " << closest(P, n) << endl;
return 0;
}

```

Trả lời: Dán kết quả thực thi vào bên dưới:

```

D:\ToanUngDungCNTT\CBThi
The smallest distance is 1.41421
-----
Process exited after 0.03071 seconds with return value 0
Press any key to continue . . .

```

Câu 4 (2 điểm): Cho hàm số $f(x) = 2e^{x^5-x^3} - 5x^3 - x + \ln(x+45) + 20$

a) (1 điểm) Trình bày thuật toán Gradient Descent để tối ưu hàm $f(x)$

Trả lời (dán sơ đồ khối hoặc ngôn ngữ tự nhiên vào bên dưới):

Input: Hàm số $f(x)$, điểm khởi đầu x_0 , hệ số động lượng (momentum coefficient) α , tham số học (learning rate) γ , số bước lặp N và sai số ε .

Output: giá trị bé nhất của $f(x)$ sử dụng phương pháp *Gradient Descent* với *Momentum*

Bước 1: Tính đạo hàm $f'(x)$

Bước 2: cho vòng lặp `for(int i = 0; i <= step; i++)`

Bước 2.1: `double tmp = gamma * (gradient(x)) + alpha * (x - prev_x);`

Bước 2.2: `if (fabs(tmp) > esp) x -= tmp;`

Bước 2.3: `prev_x = x;`

}

Bước 3: Trả về giá trị nhỏ nhất của hàm $f(x)$ với x tương ứng.

- b) (1 điểm) Viết chương trình (có dùng hàm) tính giá trị bé nhất của $f(x)$ sử dụng phương pháp *Gradient Descent* với *Momentum*, biết rằng: hệ số động lượng (momentum coefficient) α , tham số học (learning rate) γ , số bước lặp N và sai số ε .

Trả lời: Dán code vào bên dưới:

```
#include <bits/stdc++.h>

using namespace std;

const double gamma = 0.01;
const double alpha = 0.1;
const double esp = 1e-5;
const int step = 10000;

template <typename T>
T sqr(T x)
{
    return x * x;
}

template <typename T>
void get(T &x, string name)
{
    cout << "Input " << name << ": ";
    cin >> x;
}
```

```

template <typename T>
void print(T x, string name)
{
    cout << "Value of " << name << ": " << x << "\n";
}

double gradient(double x)
{
    double calc = 2*(5*pow(x,4)-3*pow(x,2))*(exp(pow(x,5)-pow(x,3))) - 15*x*x - 1 +
1.0/(x+45);
//double calc = 2*(exp(pow(x,5)-pow(x,3))) - 5*pow(x,3) - x + log(x+5) + 20;
    return calc;
}

int main(int argc, char const *argv[])
{
    double x, prev_x = 0;
    get(x, "x");
    print(gradient(x), "gradient");
    for (int i = 0; i <= step; i++)
    {
        double tmp = gamma * (gradient(x)) + alpha * (x - prev_x);
        if (fabs(tmp) > esp)
            x -= tmp;
        prev_x = x;
    }
    print(x, "x");
    return 0;
}

```

Trả lời: Dán kết quả thực thi với điểm khởi $x = 0$, $\alpha = 0.1$, $\gamma = 0.01$, $N = 10000$ và $\varepsilon = 10^{-5}$:

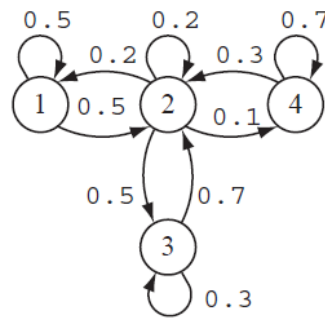
```

D:\ToanUngDungCNTT\CBThi x + v
Input x: 0
Value of gradient: -0.977778
Value of x: 1.18456

-----
Process exited after 1.434 seconds with return value 0
Press any key to continue . . . |

```

Câu 5 (1 điểm): Một hệ thống có chế độ làm việc ở mỗi giai đoạn vận hành chỉ với các trạng thái từ 1 đến 4. Chế độ làm việc của hệ thống này được mô tả bằng chuỗi Markov như hình vẽ.



a) (0.5 điểm) Xác định ma trận chuyển đổi trạng thái **P** của hệ.

Trả lời: dán kết quả vào bên dưới:

	1	2	3	4
1	0.5	0.5	0	0
2	0.2	0.2	0.5	0.1
3	0	0.7	0.3	0
4	0	0.3	0	0.7

b) (0.5 điểm) Viết chương trình (có dùng hàm) tìm xác suất (lớn nhất) khi hệ thống làm việc ở trạng thái 3 sau ba giai đoạn vận hành biết rằng hệ thống bắt đầu làm việc ở trạng thái 1.

Trả lời: Dán code vào bên dưới:

```

#include <iostream>

#include <vector>

using namespace std;

#define float_vec vector<float>

vector<float_vec> multiply(vector<float_vec> A, vector<float_vec> B, int N)
{
    vector<float_vec> C(N, float_vec(N, 0));

```

```

    for (int i = 0; i < N; ++i)
        for (int j = 0; j < N; ++j)
            for (int k = 0; k < N; ++k)
                C[i][j] += A[i][k] * B[k][j];
    return C;
}

vector<float_vec> matrix_power(vector<float_vec> M, int p, int n)
{
    vector<float_vec> A(n, float_vec(n, 0));
    for (int i = 0; i < n; ++i)
        A[i][i] = 1;
    while (p)
    {
        if (p % 2)
            A = multiply(A, M, n);
        M = multiply(M, M, n);
        p /= 2;
    }
    return A;
}

float calc_prob(vector<float_vec> M, int N, int F, int S, int T)
{
    vector<float_vec> matrix_t = matrix_power(M, T, N);
    return matrix_t[S - 1][F - 1];
}

int main()
{
    vector<float_vec> G{
        {0.5, 0.5, 0, 0},
        {0.2, 0.2, 0.5, 0.1},
    }
}

```

```

        {0, 0.7, 0.3, 0},
        {0, 0.3, 0, 0.7}}});

// so luong trang thai o moi giai doan
int N = 4;

// s: trang thai bat dau, F: trang thai ket thuc, T: so giai doan
int S = 0, F = 2, T = 3;

cout << "Max probability in state " << F+1 << ", after "
<< T << " time, start from state " << S+1 << " is: " << calc_prob(G, N, F, S, T) << endl;

return 0;

}

```

Trả lời: Dán kết quả tính toán vào bên dưới:

```

{ .\markov_chain }
Max probability in state 3, after 3 time, start from state 1 is: 7.73517e-43
PS D:\newcode\Toan_UDCNTT\11-04-2023\markov_chain>

```